

Machine Learning Project Report

Logistic Regression: “Will the client include the sauce?”

Team: Bucur Robert, Hariton Cosmin

January 2026

Contents

1	Understanding the dataset	2
1.1	Raw data description	2
1.2	Processing and Feature Engineering	2
2	Task: Logistic Regression (Binary Classification)	3
2.1	Algorithm Implementation	3
2.2	Theoretical and Experimental Justifications	3
2.3	Experimental Results	4
2.3.1	Training Evolution	4
2.3.2	Performance Metrics	4
2.3.3	Coefficient Analysis	4
3	Algorithm Comparison	5
3.1	Comparative Experimental Results	5
3.2	Performance Interpretation	5
4	Conclusions	5

1 Understanding the dataset

1.1 Raw data description

The initial dataset is a transactional dataset, with approximately 28.000 records, where each line represents a scanned product for a receipt. The principal attributes that have been selected for our analysis are:

- **id_bon**: The unique identifier for a transaction (primary key).
- **data_bon**: The transaction's timestamp, used for the processing of the temporal context.
- **retail_product_name**: The product's name, which is used as foundation for building the feature vector.
- **SalePriceWithVAT**: The price used for computing the total value of a basket.

1.2 Processing and Feature Engineering

In order to be able to apply Machine Learning algorithms, we converted the data from "long" format to "wide" format (tabular), where each instance represents an unique receipt ($N \approx 7869$ receipts).

Input Matrix X was built in the following way:

1. **Product Vectorization (One-Hot/Count Encoding)**: Each unique product becomes a distinct column. This value represents the number of apparitions of the product on the receipt.
2. **Temporal Features**: From `data_bon` we extracted:
 - **hour**: numerical value for the transaction's hour.
 - **day_of_week**: numerical value for the the transaction's day (0 for Monday to 6 for Sunday).
 - **is_weekend**: binary variable (1 for Saturday/Sunday, 0 for the others).
3. **Agregations**:
 - **total_value**: numerical value for the receipt's monetary sum.
 - **cart_size**: total number of products in the basket.

For our task (**Task 1**), we filtered the dataset and kept only the receipts that include the product "Crazy Schnitzel" and we eliminated the target value, "Crazy Sauce" to avoid data leakage. The data was normalized to assure the convergence of the optimization algorithm, using Standard Scaling. This technique is important for preprocessing data; it transforms values such that their mean is 0 and the standard deviation is 1, thus helping Gradient Descent to reach the global minimum in faster and a more stable way.

2 Task: Logistic Regression (Binary Classification)

2.1 Algorithm Implementation

We implemented the algorithm for Logistic Regression from scratch in Python, using vectorial calculations with NumPy. The used mathematical model consists of the following:

Sigmoid Function (Hypothesis):

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}}, \quad \text{unde } z = \theta^T x + b$$

Cost/Loss Function (Binary Cross Entropy): The training's goal is minimization of Negative Log-Likelihood function:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Optimization: We used **Gradient Descent** for the weights update:

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

where the learning rate α was set to 0.1, and number of iterations to 2000.

2.2 Theoretical and Experimental Justifications

This task uses (as main requirement) Logistic Regression for the following reasons:

- **Interpretation (White-box):** Unlike complex ("black-box") models like Neural Networks, Logistic Regression lets us analyze the learned coefficients (θ) in order to understand exactly which products have a good or bad influence in the decision of buying them. This direct interpretability is crucial for business insights.
- **Handling Sparse Data:** Our input matrix is relatively sparse (in general, most customers only buy a few items out of the large number of available products). Logistic Regression performs well in high-dimensional sparse settings, unlike algorithms that might suffer from the "Curse of Dimensionality" (like KNN).
- **Linear Decision Boundaries:** Shopping baskets analysis often tracks additive patterns (e.g. one item adds to the probability of sauce being bought, independently of the presence of a drink). Logistic Regression captures effectively these additive linear relations.
- **Convexity:** The cost function is convex, thus guaranteeing that Gradient Descent will converge to the global minimum; in other words, we avoid the issue of getting stuck in a local minimum (which is pretty common with more complex non-linear models).
- **Simplicity and Efficiency:** Logistic Regression is a robust model, for medium-sized datasets. If the number of features is not excessively large, the risk of overfitting is low. It's computationally efficient to train and extremely fast at inference time.

2.3 Experimental Results

2.3.1 Training Evolution

The graph below illustrates the decrease of the cost function over epochs, proving the correct convergence of the implemented algorithm.

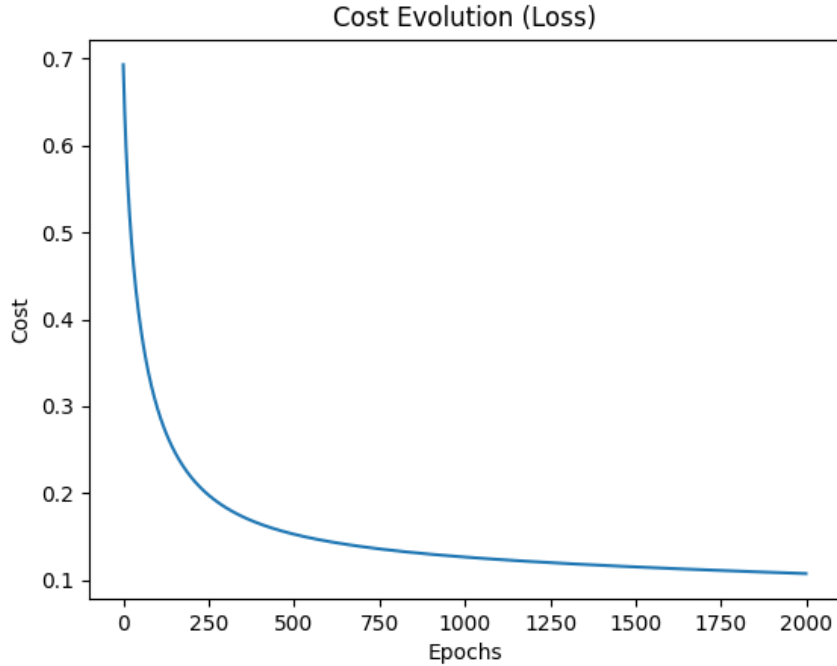


Figure 1: Cost function evolution on the training set

2.3.2 Performance Metrics

The dataset was split in 80% training instances and 20% test instances. Furthermore, before splitting them, they were chronologically sorted, such that we can train on older months and test on newer ones.

Metric	Value (Test)
Accuracy	0.9356
Precision	0.9349
Recall	0.9294
F1 Score	0.9322

Table 1: Performance of the model

2.3.3 Coefficient Analysis

By analyzing the learned weights, we identified the determining factors:

- **Positive Factors:** Products like *Pepsi*, *Baked Potatoes* or *Mac n' Cheese* and *cart_size*, *total_value* have the largest positive coefficients, indicating a strong correlation with Crazy Sauce purchase. For the last 2 features, it makes a lot of sense

in a real-world scenario, since a larger cart size increase the chances of wanting to buy a sauce for the menu, and a larger cart size means also a bigger price.

- **Negative Factors:** Products like *Spicy Sauce* or *Tomato Sauce* have strongly negative coefficients. This confirms the "substitution" hypothesis - customers rarely buy two or more types of sauce simultaneously.

3 Algorithm Comparison

To validate the performance of the primary Logistic Regression model, we implemented and tested two additional algorithms: Naive Bayes and a Decision Tree (ID3). All models were evaluated on the same preprocessed dataset, using a consistent test set of 357 samples.

3.1 Comparative Experimental Results

The table below summarizes the performance metrics obtained during the testing phase:

Algorithm	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.9356	0.9349	0.9294	0.9322
Decision Tree (ID3)	0.8739	0.8205	0.9412	0.8767
Naive Bayes	0.7395	0.6546	0.9588	0.7780

Table 2: Comparative performance of the implemented models

3.2 Performance Interpretation

- **Logistic Regression** achieved the best balance between precision and sensitivity. This confirms that the decision to include the sauce is influenced in a linear manner by the other products in the basket.
- **Decision Tree (ID3)** showed a high Recall (0.9412), but lower Precision. This suggests that while the tree captures most customers who want the sauce, it generates more false positives due to rigid decision boundaries.
- **Naive Bayes** recorded the highest Recall (0.9588) but the lowest overall accuracy. The assumption of feature independence ignores the natural correlations between complementary products.

4 Conclusions

This practical assignment has the role to prove that a simple linear algorithm like Logistic Regression, implemented from scratch, can achieve robust results on a transactional dataset. At the same time, it offers decision transparency, through coefficient interpretation, which is essential in a real-world business context.

From a **technical perspective**, implementing Logistic Regression from scratch highlighted the critical importance of preprocessing. Specifically, we observed that without

normalizing features like *total_value* and *cart_size* (i.e. using Standard Scaling), the Gradient Descent algorithm failed to converge due to the problems caused by disparate feature scales. The successful convergence of the loss curve validates the correctness of our vectorized implementation.

In conclusion, the Logistic Regression model proved to be an efficient and interpretable baseline, capable of capturing the linear additive relationships inherent in shopping basket data.